

Nama : Deswita Khansa Rafifah

NIM : 254107020151

Kelas : TI-1G

LAPORAN JOBSHEET 7

6.2. Percobaan 1 - Searching/ Pencarian Menggunakan Algoritma Sequential Search

Kode program:

> Kode program class MahasiswaBerprestasi06.java

```
package Praktikum05;

public class MahasiswaBerprestasi06 {
    Mahasiswa06 [] listMhs = new Mahasiswa06 [5];
    int idx;

    void tambah (Mahasiswa06 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("data sudah penuh");
        }
    }

    void tampil () {
        for (Mahasiswa06 m : listMhs) {
            m.tampilInformasi();
            System.out.println("-----");
        }
    }

    void bubbleSort() {
        for (int i = 0; i < listMhs.length - 1; i++) {
            for (int j = 1; j < listMhs.length - i; j++) {
                if (listMhs[j].ipk > listMhs[j-1].ipk) {
                    Mahasiswa06 tmp = listMhs[j];
                    listMhs[j] = listMhs[j-1];
                    listMhs[j-1] = tmp;
                }
            }
        }
    }
}
```

```

// tambahan kode program (Selection Sort)
void selectionSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa06 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

// tambahan kode program (Insertion Sort)
void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa06 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j - 1].ipk < temp.ipk) {
            listMhs[j] = listMhs[j - 1];
            j--;
        }
        listMhs[j] = temp;
    }
}

int sequentialSearching(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk == cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos != -1) {
        System.out.println("data mahasiswa dengan IPK : " + x + "
ditemukan pada indeks " + pos);
    }
}

```

```

    }
    else {
        System.out.println("data " + x + "tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    }
    else {
        System.out.println("Data mahasiswa dengan IPK " + x + "
tidak ditemukan");
    }
}
}

```

> Kode program class MahasiswaDemo06.java

```

package Praktikum05;
import java.util.Scanner;

public class MahasiswaDemo06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        MahasiswaBerprestasi06 list = new MahasiswaBerprestasi06();
        int jmlMhs = 5;

        for (int i = 0; i < jmlMhs; i++) {
            System.out.println("Masukkan data mahasiswa ke-" + (i +
1));

            System.out.print("Nama : ");
            String nama = sc.nextLine();
            System.out.print("NIM : ");
            String nim = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

```

```

        Mahasiswa06 m = new Mahasiswa06(nim, nama, kelas, ipk);
        list.tambah(m);
        System.out.println();
    }

    System.out.println("Data mahasiswa sebelum sorting: ");
    list.tampil();

    System.out.println("Data mahasiswa setelah sorting berdasarkan
IPK (DESC) : ");
    list.bubbleSort();
    list.tampil();

    // tambahan kode program (Selection Sort)
    System.out.println("Data yang sudah terurut menggunakan
SELECTION SORT (ASC)");
    list.selectionSort();
    list.tampil();

    // tambahan kode program (Insertion Sort)
    System.out.println("Data yang sudah terurut menggunakan
INSERTION SORT (DESC)");
    list.insertionSort();
    list.tampil();

    list.tampil();
    //melakukan pencarian data sequential
    System.out.println("-----");
    System.out.println("Pencarian data");
    System.out.println("-----");
    System.out.println("Masukkan ipk mahasiswa yang dicari: ");
    System.out.print("IPK: ");
    double cari = sc.nextDouble();

    System.out.println("menggunakan sequential searching");
    double posisi = list.sequentialSearching(cari);
    int pss = (int)posisi;
    list.tampilPosisi(cari, pss);
    list.tampilDataSearch(cari, pss);
}
}

```

Hasil running:

```
Masukkan data mahasiswa ke-1
Nama : Adi
NIM : 111
Kelas : 2
IPK : 3,6
```

```
Masukkan data mahasiswa ke-2
Nama : tio
NIM : 222
Kelas : 2
IPK : 3,8
```

```
Masukkan data mahasiswa ke-3
Nama : Ila
NIM : 333
Kelas : 2
IPK : 3,0
```

```
Masukkan data mahasiswa ke-4
Nama : Lia
NIM : 444
Kelas : 2
IPK : 3,5
```

```
Masukkan data mahasiswa ke-5
Nama : Fia
NIM : 555
Kelas : 2
IPK : 3,6
```

```
-----
Pencarian data
-----
```

```
Masukkan ipk mahasiswa yang dicari:
IPK: 3,5
menggunakan sequential searching
data mahasiswa dengan IPK :3.5 ditemukan pada indeks 3
nim      : 444
nama     : Lia
kelas    : 2
ipk      : 3.5
```

```
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD>
```

Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!

Jawab:

- Method tampilPosisi digunakan untuk menampilkan informasi mengenai letak atau indeks data yang ditemukan dalam array. Output yang dihasilkan berupa pesan yang menunjukkan posisi data berdasarkan nilai yang dicari.
- Method tampilDataSearch digunakan untuk menampilkan data lengkap mahasiswa yang ditemukan, meliputi atribut seperti NIM, nama, kelas, dan IPK.

Dengan demikian, tampilPosisi berfokus pada lokasi data, sedangkan tampilDataSearch berfokus pada isi atau detail data.

2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

Jawab: Perintah break berfungsi untuk menghentikan proses perulangan (loop) secara langsung ketika data yang dicari telah ditemukan.

Penggunaan break bertujuan untuk:

- Menghindari proses pencarian yang tidak diperlukan setelah data ditemukan.
- Meningkatkan efisiensi program.
- Mencegah terjadinya penimpaan nilai indeks jika terdapat data yang sama.

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?

Jawab: Variabel pos berfungsi untuk menyimpan indeks posisi data yang ditemukan dalam array. Nilai dari pos digunakan sebagai penghubung antara proses pencarian dengan proses penampilan hasil. Jika nilai pos = -1, maka data tidak ditemukan. Sebaliknya, jika $\text{pos} \geq 0$, maka data dapat diakses melalui listMhs[pos].

4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan.

Jawab: Jika terdapat lebih dari satu data dengan nilai yang sama, maka algoritma sequential search akan menampilkan data yang pertama kali ditemukan (indeks terkecil). Hal ini terjadi karena adanya perintah break, yang menyebabkan perulangan berhenti saat data pertama yang sesuai ditemukan, sehingga data selanjutnya tidak diperiksa.

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?

Jawab: Jika perintah break dihapus Loop akan terus berjalan sampai elemen terakhir array. Jika ada beberapa data dengan IPK yang sama, nilai posisi akan terus ditimpa setiap kali ketemu kecocokan, sehingga yang tersimpan akhirnya adalah indeks terakhir yang IPK-nya cocok (bukan yang pertama). Efisiensi juga menurun karena loop tidak berhenti meskipun data sudah ditemukan.

6.3. Percobaan 2 - Searching/ Pencarian Menggunakan Algoritma Binary Search

Kode program:

> Kode program class MahasiswaBerprestasi06.java

```
package Praktikum05;

public class MahasiswaBerprestasi06 {
    Mahasiswa06 [] listMhs = new Mahasiswa06 [5];
    int idx;

    void tambah (Mahasiswa06 m) {
        if (idx < listMhs.length) {
            listMhs[idx] = m;
            idx++;
        } else {
            System.out.println("data sudah penuh");
        }
    }
}
```

```

    }
}

void tampil () {
    for (Mahasiswa06 m : listMhs) {
        m.tampilInformasi();
        System.out.println("-----");
    }
}

void bubbleSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        for (int j = 1; j < listMhs.length - i; j++) {
            if (listMhs[j].ipk > listMhs[j-1].ipk) {
                Mahasiswa06 tmp = listMhs[j];
                listMhs[j] = listMhs[j-1];
                listMhs[j-1] = tmp;
            }
        }
    }
}

// tambahan kode program (Selection Sort)
void selectionSort() {
    for (int i = 0; i < listMhs.length - 1; i++) {
        int idxMin = i;
        for (int j = i + 1; j < listMhs.length; j++) {
            if (listMhs[j].ipk < listMhs[idxMin].ipk) {
                idxMin = j;
            }
        }
        Mahasiswa06 tmp = listMhs[idxMin];
        listMhs[idxMin] = listMhs[i];
        listMhs[i] = tmp;
    }
}

// tambahan kode program (Insertion Sort)
void insertionSort() {
    for (int i = 1; i < listMhs.length; i++) {
        Mahasiswa06 temp = listMhs[i];
        int j = i;
        while (j > 0 && listMhs[j - 1].ipk < temp.ipk) {

```

```

        listMhs[j] = listMhs[j - 1];
        j--;
    }
    listMhs[j] = temp;
}

}

int sequentialSearching(double cari) {
    int posisi = -1;
    for (int j = 0; j < listMhs.length; j++) {
        if (listMhs[j].ipk==cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}

void tampilPosisi(double x, int pos) {
    if (pos!= -1) {
        System.out.println("data mahasiswa dengan IPK :" + x + "
ditemukan pada indeks " + pos);
    }
    else {
        System.out.println("data " + x + "tidak ditemukan");
    }
}

void tampilDataSearch(double x, int pos) {
    if (pos != -1) {
        System.out.println("nim\t : " + listMhs[pos].nim);
        System.out.println("nama\t : " + listMhs[pos].nama);
        System.out.println("kelas\t : " + listMhs[pos].kelas);
        System.out.println("ipk\t : " + x);
    }
    else {
        System.out.println("Data mahasiswa dengan IPK " + x + "
tidak ditemukan");
    }
}

int findBinarySearch(double cari, int left, int right) {
    int mid;

```



```

        if (right >= left) {
            mid = (left + right)/2;
            if (cari == listMhs[mid].ipk) {
                return (mid);
            }
            else if (listMhs[mid].ipk>cari) {
                return findBinarySearch(cari, left, mid-1);
            }
            else {
                return findBinarySearch(cari, mid+1, right);
            }
        }
        return -1;
    }
}

```

> Kode program class MahasiswaDemo06.java

```

package Praktikum05;
import java.util.Scanner;

public class MahasiswaDemo06 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        MahasiswaBerprestasi06 list = new MahasiswaBerprestasi06();
        int jmlMhs = 5;

        for (int i = 0; i < jmlMhs; i++) {
            System.out.println("Masukkan data mahasiswa ke-" + (i +
1));

            System.out.print("Nama   : ");
            String nama = sc.nextLine();
            System.out.print("NIM    : ");
            String nim = sc.nextLine();
            System.out.print("Kelas : ");
            String kelas = sc.nextLine();
            System.out.print("IPK    : ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa06 m = new Mahasiswa06(nim, nama, kelas, ipk);
            list.tambah(m);
            System.out.println();
        }
    }
}

```

```

    }

    System.out.println("Data mahasiswa sebelum sorting: ");
    list.tampil();

    System.out.println("Data mahasiswa setelah sorting berdasarkan
IPK (DESC) : ");
    list.bubbleSort();
    list.tampil();

    // tambahan kode program (Selection Sort)
    System.out.println("Data yang sudah terurut menggunakan
SELECTION SORT (ASC)");
    list.selectionSort();
    list.tampil();

    // tambahan kode program (Insertion Sort)
    System.out.println("Data yang sudah terurut menggunakan
INSERTION SORT (DESC)");
    list.insertionSort();
    list.tampil();

    list.selectionSort();
    list.tampil();
    // melakukan pencarian data Binary
    System.out.println("-----");
    System.out.println("Pencarian data");
    System.out.println("-----");
    System.out.println("Masukkan ipk mahasiswa yang dicari: ");
    System.out.print("IPK: ");
    double cari = sc.nextDouble();

    System.out.println("-----");
    System.out.println("Menggunakan binary search");
    System.out.println("-----");
    double posisi2 = list.findBinarySearch(cari, 0, jmlMhs-1);
    int pss2 = (int)posisi2;
    list.tampilPosisi(cari, pss2);
    list.tampilDataSearch(cari, pss2);

    }
}

```

Hasil running:

```
Masukkan data mahasiswa ke-1
Nama : Adi
NIM : 111
Kelas : 2
IPK : 3,1

Masukkan data mahasiswa ke-2
Nama : Ila
NIM : 222
Kelas : 2
IPK : 3,2

Masukkan data mahasiswa ke-3
Nama : Lia
NIM : 333
Kelas : 2
IPK : 3,3

Masukkan data mahasiswa ke-4
Nama : Susi
NIM : 444
Kelas : 2
IPK : 3,5

Masukkan data mahasiswa ke-5
Nama : Anita
NIM : 555
Kelas : 2
IPK : 3,7

-----
Pencarian data
-----
Masukkan ipk mahasiswa yang dicari:
IPK: 3,7
-----
Menggunakan binary search
-----
data mahasiswa dengan IPK :3.7 ditemukan pada indeks 4
nim : 555
nama : Anita
kelas : 2
ipk : 3.7
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD>
```

Pertanyaan

1. Tunjukkan pada kode program yang mana proses *divide* dijalankan!

Jawab: Proses *divide* pada binary search terjadi saat array dibagi menjadi dua bagian. Hal ini ditunjukkan pada baris:

```
mid = (left + right)/2;
```

Baris tersebut digunakan untuk menentukan posisi tengah (*mid*) yang menjadi dasar pembagian array menjadi bagian kiri dan kanan.

2. Tunjukkan pada kode program yang mana proses *conquer* dijalankan!

Jawab: Proses *conquer* terjadi saat algoritma memilih bagian array yang akan diproses selanjutnya, yaitu bagian kiri atau kanan. Hal ini ditunjukkan pada bagian:

```
else if (listMhs[mid].ipk > cari) {
    return findBinarySearch(cari, left, mid-1);
}
else {
    return findBinarySearch(cari, mid+1, right);
}
```

3. Apa fungsi left, right, dan mid?

Jawab:

- left → Menunjukkan indeks terendah atau batas awal dari rentang pencarian yang sedang diperiksa. Nilainya akan berubah menjadi mid + 1 jika data yang dicari lebih besar dari nilai tengah.
- right → Menunjukkan indeks tertinggi atau batas akhir dari rentang pencarian. Nilainya akan berubah menjadi mid - 1 jika data yang dicari lebih kecil dari nilai tengah.
- mid → Menunjukkan titik tengah yang dihitung dari $(\text{left} + \text{right}) / 2$. Indeks inilah yang menjadi penentu (acuan) apakah pencarian sudah selesai (ketemu), atau harus berlanjut ke sebelah kiri/kanan.

4. Jika data IPK yang dimasukkan tidakurut. Apakah program masih dapat berjalan? Mengapa demikian?

Jawab: Program tetap dapat dijalankan, tetapi hasil pencarian menggunakan binary search tidak akan akurat atau bisa salah. Hal ini karena binary search mensyaratkan data dalam keadaan terurut. Jika data tidak terurut, maka proses pembagian array tidak akan menghasilkan pencarian yang benar.

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary search agar hasilnya sesuai

Jawab: Jika data diurutkan secara descending, maka hasil binary search tidak sesuai apabila menggunakan logika standar (ascending). Untuk memperbaikinya, kondisi perbandingan harus dibalik, yaitu:

```
else if (listMhs[mid].ipk < cari) {
    return findBinarySearch(cari, left, mid-1);
}
else {
    return findBinarySearch(cari, mid+1, right);
}
```

Hasil running:

```
-----
Pencarian data
-----
Masukkan ipk mahasiswa yang dicari:
IPK: 3,2
-----
Menggunakan binary search
-----
data mahasiswa dengan IPK :3.2 ditemukan pada indeks 4
nim      : 555
nama     : adi
kelas    : 2
ipk      : 3.2
```

```

Masukkan data mahasiswa ke-1
Nama : anita
NIM : 111
Kelas : 2
IPK : 3,8

Masukkan data mahasiswa ke-2
Nama : ila
NIM : 222
Kelas : 2
IPK : 3,7

Masukkan data mahasiswa ke-3
Nama : lia
NIM : 333
Kelas : 2
IPK : 3,5

Masukkan data mahasiswa ke-4
Nama : susi
NIM : 444
Kelas : 2
IPK : 3,4

Masukkan data mahasiswa ke-5
Nama : adi
NIM : 555
Kelas : 2
IPK : 3,2

```

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.

Jawab: Binary search menyatakan data tidak ditemukan ketika nilai left lebih besar dari right. Kondisi ini menunjukkan bahwa seluruh elemen yang mungkin telah diperiksa, namun tidak ada yang sesuai dengan nilai yang dicari. Pada kondisi tersebut, method akan mengembalikan nilai -1 sebagai tanda bahwa data tidak ditemukan.

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.

Jawab:

```

System.out.print(s: "Masukkan jumlah mahasiswa: ");
int jmlMhs = sc.nextInt();
sc.nextLine();

MahasiswaBerprestasi06 list = new MahasiswaBerprestasi06();
list.listMhs = new Mahasiswa06[jmlMhs];

```

Hasil running:

```
Masukkan jumlah mahasiswa: 5
Masukkan data mahasiswa ke-1
Nama : anita
NIM : 111
Kelas : 2
IPK : 3,8

Masukkan data mahasiswa ke-2
Nama : ila
NIM : 222
Kelas : 2
IPK : 3,7

Masukkan data mahasiswa ke-3
Nama : lia
NIM : 333
Kelas : 2
IPK : 3,5

Masukkan data mahasiswa ke-4
Nama : susi
NIM : 444
Kelas : 2
IPK : 3,4

Masukkan data mahasiswa ke-5
Nama : adi
NIM : 555
Kelas : 2
```

```
-----
Pencarian data
-----
Masukkan ipk mahasiswa yang dicari:
IPK: 3,2
-----
Menggunakan binary search
-----
data mahasiswa dengan IPK :3.2 ditemukan pada indeks 4
nim      : 555
nama     : adi
kelas    : 2
ipk      : 3.2
PS C:\Users\Asus\Documents\Algoritma dan Struktur Data\PraktikumASD> █
```